

Resolución del Práctico 1

Testing de Software 2026

Tomás Rodeghiero

Índice

1. Ejercicio 1	3
2. Ejercicio 2	3
2.1. Diferencia entre <i>fault</i> y <i>failure</i>	3
3. Ejercicio 3	4
3.1. Programa 1: <code>findLast(int[] x, int y)</code>	4
3.2. Programa 2: <code>lastZero(int[] x)</code>	4
3.3. Programa 3: <code>countPositive(int[] x)</code>	5
3.4. Programa 4: <code>oddOrPos(int[] x)</code>	5
3.5. Síntesis del ejercicio	6
4. Ejercicio 4	6
4.1. Reparaciones aplicadas	6
4.2. Tests JUnit	6
4.3. Análisis RIPR	7
4.4. Ejecución de pruebas	7
5. Ejercicio 5	8
5.1. Análisis de <code>Point</code>	8
5.2. Análisis de <code>ColorPoint</code>	9
6. Ejercicio 6	10
6.1. a) Tests, falla y corrección	10
6.2. b) Arrange–Act–Assert	11
6.3. Ejecución de pruebas	11
7. Ejercicio 7	12

7.1. Tests implementados	12
7.2. Resultado	12
7.3. Arrange-Act-Assert	12

1. Ejercicio 1

El primer ejercicio es la lectura de los capítulos 1 y 2 del libro *Introduction to Software Testing* (Ammann y Offutt, 2.^a edición).

Los conceptos más importantes son:

- **Cadena *fault* → *error* → *failure*.** El *fault* es el defecto estático en el código; el *error* es el estado interno incorrecto que se produce al ejecutarlo; la *failure* es el comportamiento externo observable. Se aplica directo en el Ejercicio 2.
- **Modelo RIPR.** Las cuatro condiciones que un test tiene que cumplir para revelar un defecto: *Reachability*, *Infection*, *Propagation* y *Revealability*. Es lo que se usa en los Ejercicios 3 y 4.
- **Criterios de cobertura.** La distinción entre el criterio (qué hay que cubrir) y los tests (los artefactos que lo cumplen). Se profundiza desde el TP3.
- **Niveles y tipos de testing.** Unidad, integración, sistema, aceptación; black-box y white-box; el lugar del testing dentro del ciclo de vida del software.

2. Ejercicio 2

2.1. Diferencia entre *fault* y *failure*

Un **fault** es la causa interna del problema: existe en el código aunque nadie lo ejecute. Es un defecto estático, sea en el código, el diseño o la especificación.

Una **failure** es la manifestación externa: solo aparece cuando el programa corre y se observa un comportamiento incorrecto desde afuera.

Entre los dos hay un concepto intermedio, el **error**, que es un estado interno incorrecto producido al ejecutar el *fault*. La relación queda:

$$fault \rightarrow error \rightarrow failure$$

Esto importa porque **no todo defecto produce una falla en toda ejecución**. Un defecto puede ejecutarse y generar un estado interno incorrecto, pero si ese estado no afecta el resultado visible, no se observa nada. Diseñar tests es, en el fondo, construir entradas que recorran ese camino completo.

Ejemplo. Un método que busca ceros en un arreglo pero arranca el recorrido desde el índice 1 en lugar de 0. Hay un *fault*. Al ejecutarlo, se genera un *error* interno. Pero si

para una entrada el resultado coincide igual con el esperado, hay fault y error pero no hay failure observable. Solo aparece la falla cuando el error logra llegar al valor devuelto.

3. Ejercicio 3

El enunciado da cuatro programas defectuosos junto con un caso de test que produce falla en cada uno. Para cada uno hay que: a) identificar el defecto y proponer una corrección; b) si es posible, una entrada que **no ejecute** el defecto; c) una entrada que lo ejecute **sin producir falla**; d) una entrada que lo ejecute **y sí produzca falla**. Los puntos b, c y d se mapean directamente sobre la cadena RIPR.

El código original vive en `DefectivePrograms.java` y la corrección en `FixedPrograms.java`.

3.1. Programa 1: `findLast(int[] x, int y)`

Defecto y corrección. El `for` itera mientras `i > 0`, así que la posición 0 nunca se inspecciona. La corrección es relajar la guarda:

```
1 // Defectuoso
2 for (int i = x.length - 1; i > 0; i--) {
3     if (x[i] == y) return i;
4 }
5
6 // Corregido
7 for (int i = x.length - 1; i >= 0; i--) {
8     if (x[i] == y) return i;
9 }
```

b) No hay entrada válida que evite ejecutar el defecto: la guarda se evalúa siempre. Lo que sí puede pasar es que se ejecute y no se note.

c) `x = [5, 2, 3]`, `y = 2`. Esperado 1. El defecto se ejecuta pero el valor buscado no está en la posición 0, así que el resultado sale bien.

d) `x = [2, 3, 5]`, `y = 2`. Esperado 0, devuelve -1. El 2 está justo en la posición que el recorrido nunca visita.

3.2. Programa 2: `lastZero(int[] x)`

Defecto y corrección. Tiene que devolver el último índice con valor 0, pero recorre de izquierda a derecha y retorna el primero. La corrección es invertir el sentido:

```

1 // Corregido
2 for (int i = x.length - 1; i >= 0; i--) {
3     if (x[i] == 0) return i;
4 }

```

b) Con arreglos no nulos no se puede evitar el recorrido defectuoso.

c) $x = [1, 0, 2]$. Esperado 1. Hay un solo cero, así que el primero y el último coinciden.

d) $x = [0, 1, 0]$. Esperado 2, devuelve 0.

3.3. Programa 3: `countPositive(int[] x)`

Defecto y corrección. El comentario dice “positivos” pero la condición es $x[i] \geq 0$, así que cuenta también al 0. La corrección es estricta:

```

1 if (x[i] > 0) {
2     count++;
3 }

```

b) $x = []$. El cuerpo del `for` no se ejecuta.

c) $x = [-4, 2, 2]$. Esperado 2. La condición defectuosa se evalúa, pero como no hay ceros, contar ≥ 0 o > 0 da lo mismo.

d) $x = [-4, 2, 0, 2]$. Esperado 2, devuelve 3. El cero se cuenta como positivo.

3.4. Programa 4: `oddOrPos(int[] x)`

Defecto y corrección. Tiene que contar impares o positivos, y usa $x[i] \% 2 == 1$. El problema es que en Java $-3 \% 2 == -1$, así que los impares negativos quedan afuera. La corrección verifica que el resto sea distinto de cero:

```

1 if (x[i] % 2 != 0 || x[i] > 0) {
2     count++;
3 }

```

b) $x = []$.

c) `x = [2, 4, -2]`. Esperado 2. La condición se evalúa pero no hay impares negativos que la disparen.

d) `x = [-3, -2, 0, 1, 4]`. Esperado 3, devuelve 2. El -3 no se cuenta.

3.5. Síntesis del ejercicio

Los cuatro defectos son del mismo estilo: errores de borde u omisiones en la condición lógica que controla el ciclo o el filtro. Resumen rápido:

- `findLast`: no revisa el índice 0.
- `lastZero`: devuelve el primer cero en vez del último.
- `countPositive`: cuenta el 0 como positivo.
- `oddOrPos`: la semántica de `%` en Java deja afuera a los impares negativos.

4. Ejercicio 4

Tomando los cuatro programas del ejercicio anterior, hice tres cosas: apliqué las reparaciones en `RepairedPrograms.java`, llevé los casos de prueba a JUnit en `RepairedProgramsTest.java`, y para cada uno argumenté por qué el test detecta la falla usando RIPR.

4.1. Reparaciones aplicadas

Cada cambio toca únicamente la condición que provocaba el problema:

- `findLast`: la guarda del ciclo pasa de `i > 0` a `i >= 0`.
- `lastZero`: se invierte el sentido del recorrido.
- `countPositive`: se reemplaza `>= 0` por `> 0`.
- `oddOrPos`: se sustituye `% 2 == 1` por `% 2 != 0`.

4.2. Tests JUnit

Reusé los casos del enunciado, que eran los que evidenciaban la falla en la versión defectuosa. Acá un par de ejemplos tal como quedaron en la suite:

```
1 @Test
```

```

2 void
    findLast_debeEncontrarElValorEnLaPrimeraPosicionSiCorresponde()
    {
3     // Si el valor buscado esta en la posicion 0, no debe
        saltarlo.
4     assertEquals(0, RepairedPrograms.findLast(new int[] {2, 3,
        5}, 2));
5 }
6
7 @Test
8 void countPositive_noDebeContarElCeroComoPositivo() {
9     // Solo los valores estrictamente > 0 cuentan como positivos.
10    assertEquals(2, RepairedPrograms.countPositive(new int[] {-4,
        2, 0, 2}));
11 }

```

Sobre `OriginalPrograms` estos casos detectan la falla; sobre `RepairedPrograms` pasan.

4.3. Análisis RIPR

findLast([2,3,5], 2). El test entra al `for` y llega a la guarda defectuosa (R). Cuando le tocaría revisar `i = 0`, esa posición no se inspecciona (I). Como el valor buscado está justo ahí, el error afecta el retorno (P). El `assertEquals` compara `-1` contra `0` y la falla queda visible (R).

lastZero([0,1,0]). El test entra al recorrido de izquierda a derecha (R). El programa toma como respuesta el primer cero, aunque hay otro más adelante (I). Ese índice se devuelve sin más (P). El test esperaba `2` y recibe `0` (R).

countPositive([-4,2,0,2]). El test ejecuta `x[i] >= 0` (R). Al llegar al `0`, el contador aumenta de más (I). Eso afecta el valor final (P). El test esperaba `2` y recibe `3` (R).

oddOrPos([-3,-2,0,1,4]). El test evalúa `x[i] % 2 == 1 || x[i] > 0` (R). Al llegar al `-3`, el método no lo cuenta aunque es impar (I). El contador queda una unidad por debajo (P). El test esperaba `3` y recibe `2` (R).

4.4. Ejecución de pruebas

Desde `tp1/ejercicio4`:

```
mvn -Dmaven.repo.local=.m2 test
```

5. Ejercicio 5

Acá se analizan dos clases del paquete `practico1_exercises.color_points`, tomadas del ejemplo de Bloch en *Effective Java* (capítulo 3, sobre el contrato de `equals`): `Point` (igualdad por valor) y `ColorPoint` (le agrega color extendiendo `Point`). El problema clásico es que extender una clase con igualdad por valor y sumarle más estado rompe la simetría y la transitividad de `equals`.

5.1. Análisis de Point

a) **Qué tiene de malo y cómo lo modificaría.** El `equals` de `Point` en sí no falla cuando se lo compara con otro `Point`. El problema es que `Point` es instanciable, no final, y su `equals` acepta cualquier instanceof `Point`:

```
1 @Override
2 public boolean equals(Object o) {
3     if (!(o instanceof Point)) {
4         return false;
5     }
6     Point p = (Point) o;
7     return p.x == x && p.y == y;
8 }
```

Una subclase como `ColorPoint` también es vista como `Point`, y entonces el `equals` ignora el color. Para evitarlo: marcar `Point` como `final`, o usar composición en vez de herencia. Lo correcto es lo segundo: que `ColorPoint` tenga un `Point` adentro, no que herede de él.

b) Comparando solo `Point` con `Point`, la falla no aparece:

```
1 Point p1 = new Point(1, 2);
2 Point p2 = new Point(1, 2);
3 assert p1.equals(p2);
```

c) Acá se ejecuta la falla pero el resultado sigue siendo correcto:

```
1 Point p = new Point(1, 2);
2 ColorPoint cp = new ColorPoint(2, 3, Color.RED);
3 assert !p.equals(cp);
```

`Point.equals` acepta comparar contra un `ColorPoint`, pero como las coordenadas ya son distintas, igual devuelve `false`.

d) No lo encuentro. Cuando aparece el error se ve directo en el valor que devuelve `equals`; no hay un estado interno oculto que pueda no propagarse.

5.2. Análisis de `ColorPoint`

a) **Qué tiene de malo y cómo lo modificaría.** Hereda de `Point` y redefine `equals` exigiendo que el otro objeto también sea `ColorPoint`:

```
1 @Override
2 public boolean equals(Object o) {
3     if (!(o instanceof ColorPoint)) {
4         return false;
5     }
6     ColorPoint cp = (ColorPoint) o;
7     return super.equals(o) && cp.color.equals(color);
8 }
```

Eso rompe la simetría. Por ejemplo:

```
1 Point p = new Point(1, 2);
2 ColorPoint cp = new ColorPoint(1, 2, Color.RED);
3
4 p.equals(cp);    // true
5 cp.equals(p);    // false
```

`p.equals(cp)` da `true` porque `Point` solo mira `x` e `y`. `cp.equals(p)` da `false` porque exige que el otro sea `ColorPoint`. Esa asimetría viola el contrato.

La solución es no heredar y usar composición. Esa es `ColorPointFixed`:

```
1 public class ColorPointFixed {
2     private final Point point;
3     private final Color color;
4
5     public ColorPointFixed(int x, int y, Color color) {
6         this.point = new Point(x, y);
7         this.color = Objects.requireNonNull(color);
8     }
9
10    @Override
11    public boolean equals(Object o) {
12        if (!(o instanceof ColorPointFixed)) {
13            return false;
14        }
15    }
```

```

15         ColorPointFixed cp = (ColorPointFixed) o;
16         return point.equals(cp.point) && color.equals(cp.color);
17     }
18 }

```

Como `ColorPointFixed` no hereda de `Point`, no entra en conflicto con su `equals`.

b) `ColorPoint` con `ColorPoint`, sin mezclar tipos: la falla no aparece.

c) Con coordenadas distintas:

```

1 ColorPoint cp = new ColorPoint(1, 2, Color.RED);
2 Point p = new Point(4, 5);
3 assert !cp.equals(p);

```

La rama defectuosa se ejecuta (`p` no es `ColorPoint`), pero el resultado sigue siendo el esperado.

d) Tampoco lo encuentro acá, por la misma razón: cuando el problema aparece, se manifiesta directo en lo que devuelve `equals`.

6. Ejercicio 6

Aclaración sobre el material. El enunciado dice que `PointTest` viene dado, pero en el material del Classroom no estaban ni la clase, ni el test, ni el paquete `practico1_exercises.point_set`. Para poder resolverlo armé mi propia `Point`, mi propia `PointTest`, y sobre eso trabajé los puntos a) y b).

6.1. a) Tests, falla y corrección

Los tests que agregué:

- igualdad entre dos puntos con las mismas coordenadas;
- desigualdad cuando alguna coordenada cambia;
- ausencia de duplicados al insertar puntos equivalentes en un `HashSet`;
- reconocimiento de un punto equivalente con `contains`.

Los dos últimos son los importantes: como el paquete se llama `point_set`, tiene sentido verificar cómo se comporta `Point` dentro de un conjunto.

Con esos tests aparece la falla clásica: si `Point` redefine `equals` pero no redefine `hashCode` de manera consistente, dos puntos iguales pueden comportarse como distintos en un `HashSet`. Eso rompe la promesa de `HashSet` y `HashMap`.

La corrección es agregar un `hashCode` consistente con `equals`:

```
1 @Override
2 public int hashCode() {
3     return Objects.hash(x, y);
4 }
```

Con eso, dos `Point` con los mismos `x` e `y` quedan iguales tanto para `equals` como para `hashCode`, y los tests sobre conjuntos pasan.

6.2. b) Arrange–Act–Assert

Dejé las tres fases marcadas con comentarios en cada test. Un ejemplo:

```
1 @Test
2 void unHashSetNoDebeGuardarDuplicadosSiLosPuntosSonIguales() {
3     // arrange: preparamos un Set y dos objetos equivalentes.
4     Set<Point> points = new HashSet<>();
5     Point p1 = new Point(1, 2);
6     Point p2 = new Point(1, 2);
7
8     // act: intentamos agregar ambos puntos.
9     points.add(p1);
10    points.add(p2);
11
12    // assert: el set debe conservar un unico elemento, sin
13        duplicados.
14    assertEquals(1, points.size());
15 }
```

6.3. Ejecución de pruebas

Desde `tp1/ejercicio6`:

```
mvn -Dmaven.repo.local=.m2 test
```

7. Ejercicio 7

Aclaración sobre el material. El enunciado menciona `PointSet` y `PointSetTest` como dadas, pero en el material no estaban. Para poder hacer el ejercicio armé las dos y los tests que cubren el comportamiento del conjunto.

7.1. Tests implementados

Los casos cubren el comportamiento básico de un conjunto de puntos:

- un `PointSet` recién creado empieza vacío;
- no se agregan duplicados cuando los puntos son equivalentes;
- `contains` reconoce un punto equivalente;
- `remove` elimina un punto equivalente;
- `contains` devuelve `false` cuando el punto no está.

7.2. Resultado

Con la implementación de `PointSet` todos los tests pasan; no encontré una falla nueva. Tiene sentido: `PointSet` se apoya en un `HashSet<Point>`, y en el ejercicio anterior ya se había corregido `Point` agregando el `hashCode` consistente con `equals`. Una vez que esas dos operaciones quedan alineadas, `HashSet` funciona como tiene que funcionar y `PointSet` hereda ese buen comportamiento sin tener que ajustar nada propio.

7.3. Arrange–Act–Assert

Cada test tiene las tres fases marcadas con comentarios. Por ejemplo, el test de borrado:

```
1 @Test
2 void removeDebeEliminarUnPuntoEquivalente() {
3     // arrange: cargamos un punto y creamos otro equivalente para
4     // remover.
5     PointSet pointSet = new PointSet();
6     Point stored = new Point(5, 6);
7     Point removed = new Point(5, 6);
8     pointSet.add(stored);
9
10    // act: removemos usando un objeto distinto pero equivalente.
11    boolean removedResult = pointSet.remove(removed);
```

```
12     // assert: la operacion debe informar exito y el conjunto
        quedar vacio.
13     assertTrue(removedResult);
14     assertEquals(0, pointSet.size());
15 }
```